第4章 软件安全
恶意代码与漏洞

内容提纲

- 恶意软件的类型
- 恶意代码分析
- 漏洞原理

软件安全

软件面临的安全威胁：

- 软件自身安全: 软件漏洞Vulnerabilities/漏洞利用exploit
- 恶意软件Malware攻击和检测
- 软件逆向分析(软件破解)与防护

造成的危害：

- 软件被恶意控制(正常功能被修改或破坏)
- 数据被窃取

早期恶意软件的分类

恶意软件

基本技术

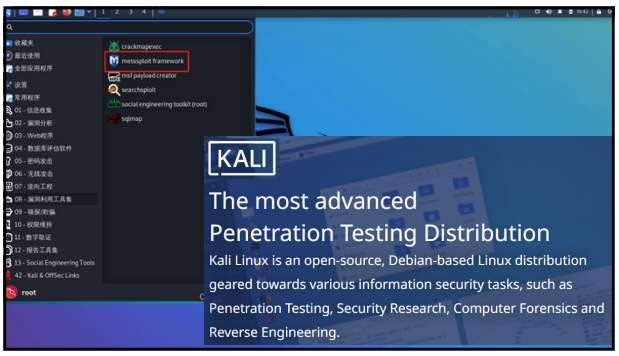
- 特洛伊木马：诱使用户进行安装
- 蠕虫+病毒：自我复制、感染

组合攻击

- 漏洞利用Exploit：利用软件漏洞获取对系统的访问权限
- 网络钓鱼：诱使用户提供可用于获取访问权限的信息
- Rootkit或Bootkit：攫取管理访问权限，以规避检测并获得更多控制权

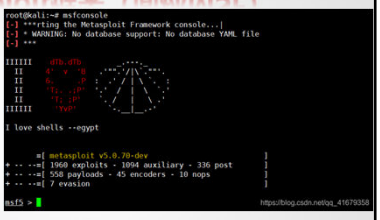
攻击目标

- 广告软件：显示令人讨厌的广告
- 僵尸网络：将设备置于外部控制之下
- 加密货币挖矿机：利用计算能力从事加密货币挖矿
- 勒索软件：索取钱财
- 间谍软件：通过键盘记录器或其它方式秘密收集数据
- 其它损坏：数据损坏、故意破坏、蓄意捣乱



Metasploit框架（简称MSF）

- 用于渗透测试的开源工具，由Ruby程序语言编写的模板化框架，具有很好的扩展性。
- 有六个模块：辅助模块(auxiliary)、渗透攻击模块(exploits）、后渗透攻击模块(post)、攻击载荷模块(payloads)、空指令模块(nops）、编码器模块(encoders)，其中msf为总模块，其他均为分支模块

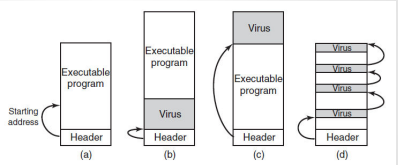


MSF对恶意软件的分类

病毒

Windows下的病毒

- Win32/64 PE病毒(C语言、汇编语言)
- UNIX/Linux环境下：ELF病毒
- 宏病毒(VBScript等)
- 脚本病毒(VB Script、Java Script、PHP等)
- ……



宏病毒

- 利用如下自动执行宏，将病毒代码写在如下宏中，由于这些宏会自动执行，因此获取控制权。

WORD	EXCEL	Office97/2000
AutoOpen	Auto_Open	Document_Open
AutoClose	Auto_Close	Document_Close
AutoExec		
AutoExit		
AutoNew		Document_New
	Auto_Activate	
	Auto_Deactivate	

脚本病毒

- 用VB Script、Java Script、PHP、JSON等脚本语言编写的恶意程序

脚本病毒的特点

- 不需要编译，方便快捷；
- 以文本保存，容易突破操作系统的安全机制
- 能够实现PE病毒几乎所有的病毒功能，但文件大小明显小于PE类病毒
- 伪装性强，混淆手法更为多变，混淆成本远低于PE类病毒（如更改文件名后缀、图标）
- 易于自动生产，容易更改配置产生新的病毒
- 编写简单，新手很容易上手
- 方便网络传播（最开始就是为方便网络编程诞生的）
  - 通过电子邮件附件传播
  - 通过局域网共享传播
  - 通过感染网页文件传播
  - 通过聊天通道传播

恶意软件

蠕虫+病毒+木马

熊猫病毒FAQ

问：什么是熊猫烧香病毒？

答："熊猫烧香"其实是一种蠕虫病毒的变种，而且是经过多次变种而来的。尼姆亚变种W(Worm.Nimaya.w)，由于中毒电脑的可执行文件会出现"熊猫烧香"图案，所以也被称为"熊猫烧香"病毒。用户电脑中毒后可能会出现蓝屏、频繁重启以及系统硬盘中数据文件被破坏等现象。同时，该病毒的某些变种可以通过局域网进行传播，进而感染局域网内所有计算机系统，最终导致企业局域网瘫痪，无法正常使用。

病毒特征

1、这个病毒关闭众多杀毒软件和安全工具

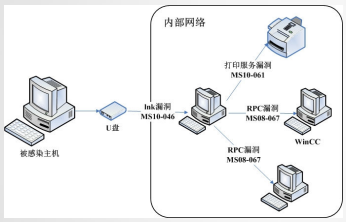
2、循环遍历磁盘目录，感染文件，对关键系统文件跳过

3、感染所有EXE、SCR、PIF、COM文件，并更改图标为烧香熊猫

4、感染所有.htm/.html/.asp/.php/.jsp/.aspx文件，添加木马恶意代码

5、自动删除*.gho文件

Stuxnet蠕虫病毒



- 专门攻击工业系统
- 利用多个零日漏洞
- 使用数字签名
- 明确的攻击目标-伊朗核设施

病毒和蠕虫的区别

表 1. 病毒和蠕虫的一些差别

	病毒	蠕虫
存在形式	寄生	独立个体
复制机制	插入到宿主程序（文件）中	自身的拷贝
传染机制	宿主程序运行	系统存在漏洞（vulnerability）
搜索机制（传染目标）	针对本地文件	针对网络上的其它计算机
触发传染	计算机使用者	程序自身
影响重点	文件系统	网络性能、系统性能
计算机使用者角色	病毒传播中的关键环节	无关
防治措施	从宿主文件中摘除	为系统打补丁（Patch）
对抗主体	计算机使用者、反病毒厂商	系统提供商、网络管理人员

木马

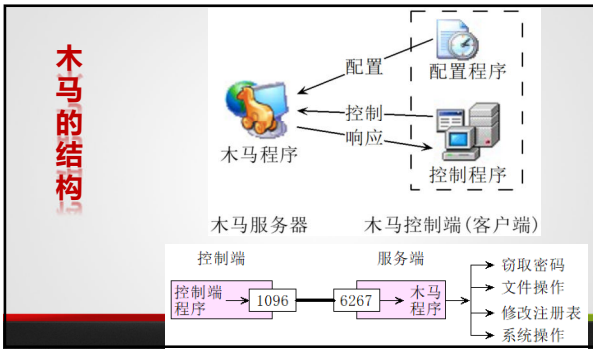
- 本质：远程控制程序

木马和合法远程控制的区别

是否可见可控

- 木马不通知用户，不经过用户允许
- 合法远程控制软件对一些敏感操作会给用户提示，请求用户授权等

木马的结构



网页木马的入侵过程

一、概述

近期，CNCERT 和安天联合监测到一批伪装成 CorelDraw、Notepad++、IDA Pro、WinHex 等多款实用软件进行传播的窃密木马。通过跟踪监测发现其每日上线境内肉鸡数（以 IP 数计算）最多已超过 1.3 万，由于该窃密木马会收集浏览器书签、邮箱账户等信息，故我们将命名为“魔盗”。

攻击者利用“cdr[.]jyxwlkj.cn”及“cdmb[.]jyxwlkj.cn”域名建立多个软件下载页面，用于投放伪装成实用软件的“魔盗”窃密木马。窃密木马运行后会收集受害者主机中已安装的软件列表与多款浏览器的历史记录、书签数据和邮件客户端邮箱账户信息，并加密回传至攻击者服务器。由于部分恶意程序具备在线升级能力，因此攻击者可随时更改攻击载荷（如勒索、挖矿、窃密等不同目的的攻击载荷），给受害者造成更大损失。

从攻击者服务器中下载窃密组件进行数据窃取并回传。

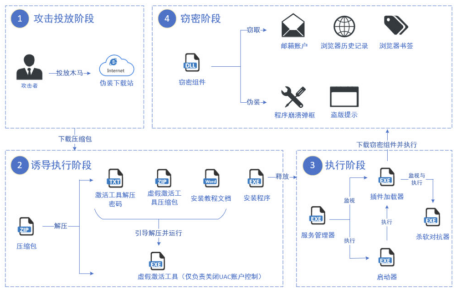
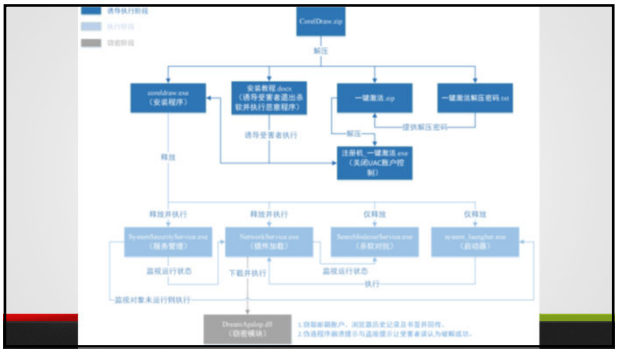
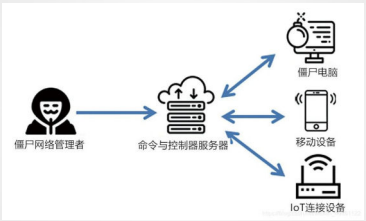


图 1 攻击流程图



僵尸Bot/僵尸网络BotNet



关于 Mirai 变种 Miori 僵尸网络大规模传播 的风险提示

- 该僵尸网络为 Mirai 变种，包括针对 mips、arm、x86 等 CPU 架构 的样本，由于该僵尸网络样本均以 miori 命名，将其命名 为 Mirai_miori。
- 在 2 个月的时间中，捕获到 Mirai_miori 僵尸网络样本至少迭代过 3 个版本，具有 9 个传播源，涉及 6 个 C2 服务器，传播方式主要为弱口令爆破以及 1 day 和 N day 漏洞。
- Mirai_miori 僵尸网络出现以来所投递的样本变动很 小，运营者将主要精力投入到漏洞搜集利用以及更换 C2 服务 器上。

1、样本运行后在控制台上输出以下内容“your device just got infected to a bootnoot”，并修改进程名为" "

```
v24 = ((int (__fastcall *)(int, int *))sub_C4F0)(1, &v67);
_libc_write(1, v24, v67);
_libc_write(1, "\n", 1);
encode(1u);
strcpy(v61, " ");
memset(&v61[3], 0, 97);
v25 = prctl(15, v61);
```

图 1 样本运行信息

2、遍历/proc/目录下的文件，结束掉指定进程。

```
_GI_strcpy(v32, "/proc/");
_GI_strcat(v32, v9);
_GI_strcat(v32, "/maps");
_GI_strcpy(v30, "/proc/");
_GI_strcat(v30, v9);
_GI_strcat(v30, "/exe");
_GI_strcpy(v33, "/proc/");
_GI_strcat(v33, v9);
_GI_strcat(v33, "/comm");
_GI_strcpy(v29, "/proc/");
_GI_strcat(v29, v9);
_GI_strcat(v29, "/cwd");
v11 = _GI_readlink(v30, v31, 63);
```

图 2 结束指定进程

之后每 60s 发送固定 2 字节心跳包\x00\x00，若 10 秒内有收到 C2 下发的非心跳指令，则心跳包间隔时间会相应增加。

```
else
{
  v40 = strlen(v43);
  LOBYTE(v40[0]) = v40;
  dword_1A024 = ((int (__fastcall *)(int))get_localaddr)(v40);
  _libc_send(dword_1A024, &unk_13190, 4, 0x4000);
  _libc_send(dword_1A024, v40, 1, 0x4000);
  if ( LOBYTE(v40[0]) )
    _libc_send(dword_1A024, v42, LOBYTE(v40[0]), 0x4000);
  }
}
while ( dword_1A024 == -1
```

图 3 上线机制

样本分析

样本分析

4、DDoS 攻击方法，样本内置了 8 种 DDoS 攻击，部分 DDoS 攻击复用了 mirai 的源码。

攻击方法名称	含义	特点
tcp_stomp	ack flood 攻击变体	高 BPS
tcp_syn	半开连接攻击，耗尽服务器资源	高 BPS
tcp_ack	在 tcp 连接建立之后，发送带有 ack 标志位的数据包。	高 BPS
udp_plain	udp flood 攻击变体	高 BPS
gre_ip	修改的 greeth flood	高 BPS
gre_eth	基于 GRE 协议的洪水攻击	高 BPS
udp_ves	基于 UDP 的洪水攻击	针对受 OVH 保护的服务器
tcp_plain	修改的 tcp flood	高 BPS
tcp_stomp	ack flood 攻击变体	高 BPS

传播方式分析

V3 版本	内容	备注
样本自传播方式	telnet 爆破 23，2323	XOR 加密，弱口令增多，密钥改为 0x3
传播源服务器攻击方式	ssh 爆破 22，2222	
	CVE-2022-22965	
	telnet 爆破 23，2323	
	NetCore 53413 后门漏洞	
	CVE-2022-29591	
	CVE-2017-9100	
	CVE-2017-17215	
	CVE-2022-29464	
	D-Link DIR-823G v1.02 B05 命令注入漏洞	
	CVE-2021-35395	
	Hadoop Yarn REST API 未授权漏洞利用	
C2	179.43.156.214	
传播源	179.43.156.214	
传播时间推测	2022 年 4 月 28 日至今	

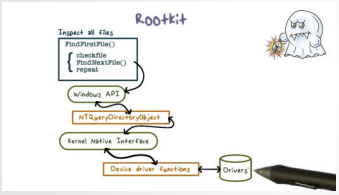
运营者利用的漏洞

漏洞	简介
CVE-2017-9100	身份认证绕过漏洞，受影响设备为固件版本为 3.04 的 D-Link DIR-600M 设备。
CVE-2017-17215	华为 HG532 部分定制版本存在远程代码执行漏洞，攻击者通过向 37215 端口发送恶意数据包，成功利用可以导致远程代码执行。
CVE-2019-11399	命令注入漏洞，受影响设备 TRENDnet TEW-651BR 2.04B1、TEW-652BRP 3.04b01 和 TEW-652BRU 1.00b12 。
CVE-2021-35395	Realtek Jungle SDK v2.x 至 v3.4.14B 版本中存在缓冲区溢出漏洞。
CVE-2022-29591	缓冲区溢出漏洞，受影响设备 Tenda TX9 Pro 22.03.02.10。
CVE-2022-22965	远程代码执行，在 JDK 9+ 上运行的 Spring MVC 或 Spring WebFlux 应用程序都可能遭受攻击。
CVE-2022-29464	未经身份验证的无限制任意文件上传漏洞，允许未经身份验证的攻击者通过上传恶意 JSP 文件在 WSO2 服务器上获得 RCE。
D-Link DIR-823G v1.02 B05 命令注入漏洞	攻击者可以通过 POST 的方式往 /HNAP1 发送精心构造的请求，执行任意的操作系统命令。

各版本弱口令解密后内容

V1		V2		V3	
root		root		root	icatch99
default		root	root	root	Zte521
admin	admin	Admin	Admin	ubnt	ubnt751t6
root	vizxv	admin	password	root	changeme
root	default	default		root	tu2@@23
ofbhome	ofbhome	ubnrT	ubnrT	default	0ab1e5c8
ofbtelnet	ofbtelnet	root	123456	default	41qqhu6
root	ttnet	user	user	default	52fGgNfs
admin	gpon	root	1234	root	
root	ste	admin	1234	root	root
telecomadmin	admintelecom	admin	admin	Admin	Admin
telnet	telnet	root	vizxv	admin	password
telnetadmin	telnetadmin	root	default	default	
support	support	ofbhome	ofbhome	ubnt	ubnt
root	1001chin	ofbtelnet	ofbtelnet	root	123456
admin	aquario	root	ttnet	user	user
default	default	admin	gpon	root	1234
adm		root	ste	admin	1234
root	taZz@23495859	telecomadmin	admintelecom	admin	admin
root	tlnginqcn	telnet	telnet	root	vizxv
admin	admin123	telnetadmin	telnetadmin	root	default
root	GM8182	support	support	ofbhome	ofbhome

Rootkit



- 小程序组成的工具包，使得攻击者能够保持访问计算机上具有最高权限的用户“root”。
- 隐身是主要目标：
  - 能够持久或可靠地、无法被检测地存在于计算机上。
  - 以隐藏恶意程序为主。

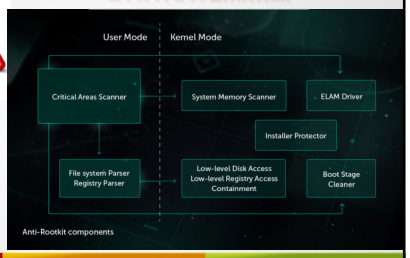
Rootkit的防御

User-Mode Rootkit

Kernel-Mode Rootkit

Hybrid Rootkit

Firmware Rootkit



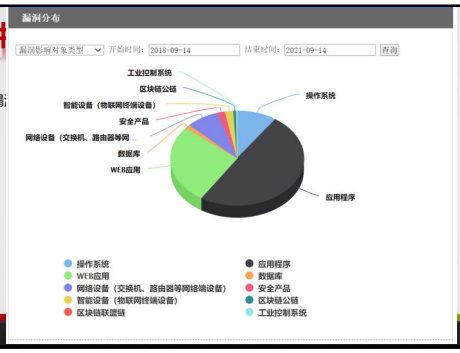
软件漏洞原理

软件漏洞

- 漏洞，也称脆弱性（Vulnerability），RFC 2828将漏洞定义为“系统设计、实现或 操作和管理中存在的缺陷或弱点，能被利用而违背系统的安全策略”。
- 漏洞是计算机系统在硬件、软件、协议的具体实现或系统策略上存在的缺陷和不足。一旦被发现，攻击者就可以利用这跟额漏洞获得计算机的额外权限，在未授权的情况下访问或破坏系统，从而危害计算机系统安全。

软件

- 漏



软件漏洞利用对系统的威胁

非法获取访问权限	绕过、突破访问控制的访问规则(不同用户组拥有不同的访问权限)	
权限提升	将当前较低的账户权限提升到更高级别的用户权限	
拒绝服务	使计算机软件或系统无法正常工作、无法提供正常的服务	本地拒绝服务漏洞 远程拒绝服务漏洞
恶意软件植入	主动植入：不需要人的干预。利用系统正常功能或缺陷漏洞将攻击代码植入目标中，，如病毒、蠕虫	
	被动植入：需要人的任何干预。物理接触目标并植入、手工植入、自己下载	
数据丢失或泄露	1. 文件访问权限设置错误；2. Web程序中，没有充分验证用户输入；3. 系统漏洞，服务器信息泄露	

典型的软件漏洞

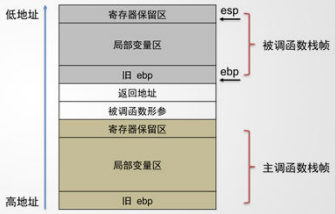
作为数据输入，作为指令执行

- 缓冲区溢出
- 注入类漏洞
  - 针对桌面软件、系统程序的二进制代码注入，程序缺陷导致控制权被劫持
  - 针对Web应用的脚本代码，在输入过滤时存在缺陷
    - ✓ SQLi
    - ✓ 操作系统命令注入
    - ✓ Web脚本语言注入
    - ✓ SOAP注入
- 权限类漏洞

典型软件漏洞机理分析

缓冲区溢出漏洞

- 栈、堆
- 栈帧
- 缓冲区溢出



栈溢出的利用

```
#include <stdio.h>
#define PASSWORD "1234567"
int verify_password (char *PASSWORD)
{
    int authenticated;
    char buffer[4];
    authenticated = strcmp(password, PASSWORD)
    strcpy(buffer,password)
    return authenticated;

}

main()
{
    int valid_flag = 0;
    char password[1024];
    while(1)
    {
        printf("please input password:        ");
        scanf("%s",password);
        valid_flag = verify_password(password);
        if (valid_flag)
        {
            printf("\"incorrect password! \n\n");

        }
        else{
            printf("Congratulation! You have passed the verification!\n");
            break;
        }
    }
}
```

- 修改邻接变量

栈帧分布状态

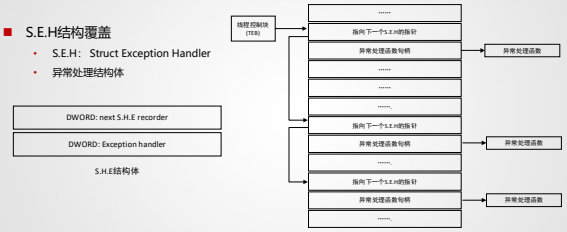
栈溢出的利用

- 修改函数返回地址

修改返回函数地址

栈溢出的利用

- S.E.H结构覆盖
  - S.E.H：Struct Exception Handler
  - 异常处理结构体



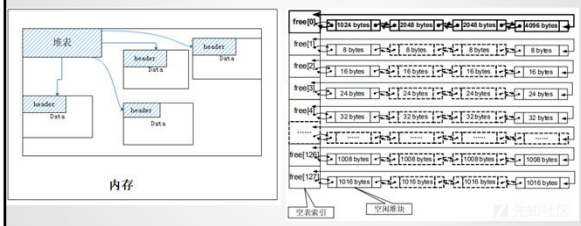
S.H.E结构体

S.E.H链表结构图

栈与堆

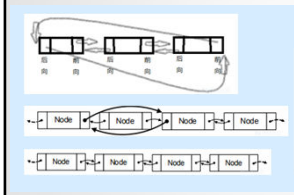
	堆内存	栈内存
典型用例	动态增长的链表等数据结构	函数局部数组
申请方式	需要函数(malloc, new)动态申请，通过返回的指针使用	在程序中直接申明即可：变量，数组，……
释放方式	需要专门的函数来释放，如free, delete	系统自动回收，栈平衡
管理方式	由程序员负责申请与释放，系统自动合并	由系统完成
所处位置	变化范围很大	一般是在0x0010xxxx
增长方向	从内存低地址向高地址排列	从内存高地址向低地址排列

堆在内存中的组织

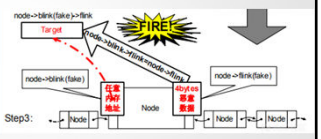


堆溢出的利用

- 正常的堆块去除操作



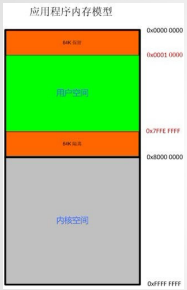
- DWORD SHOOT
  - (向内存任意位置写任意数据)



栈溢出堆溢出的防御

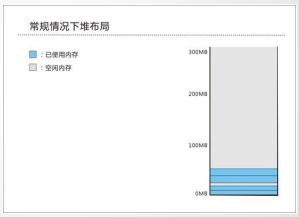
- 对于已有的攻击方法，大部分已经很难利用了

目的	保护技术
覆盖返回地址	通过GS保护
覆盖SEH链	通过SafeSEH、SEHOP保护
覆盖本地变量	可能被VC编译器经过重新整理和优化
覆盖空闲堆双向链表	通过safe unlinking保护
覆盖堆块头	XP下使用8位的HeaderCookie进行保护，VISTA之后使用XOR HeaderData
覆盖lookaside linked list	Vista之后被移除

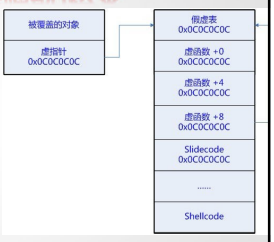


堆溢出的利用

- Heap Spray(堆喷射)技术



Heap Spray(堆喷射)技术

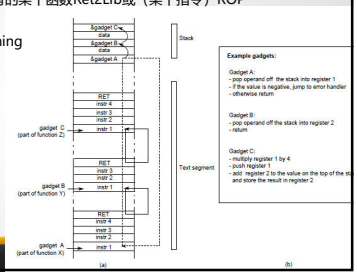


Heap Spray的优缺点

优 点	缺 点
增加缓冲区溢出攻击的成功率	会导致被攻击进程的内存占用暴增，容易被察觉
覆盖地址变得更加简单了，可以简单使用类NOP指令来进行覆盖	不能用于主动攻击，一般是通过栈溢出利用或者其他漏洞来进行协同攻击
它也可以用于堆栈溢出攻击，用slidecode覆盖堆栈返回地址即可	如果目的地址被shellcode覆盖，则shellcode执行会失败，因此不能保证100%成功

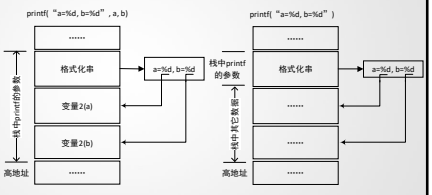
缓冲区溢出攻击的改进-Ret2Lib和ROP

- 修改返回地址，让其指向内存中已有的某个函数Ret2Lib或（某个指令）ROP
- ROP: Return Oriented Programming
- 别称：Code Reuse Attacks



格式化串漏洞

```
int main(void)
{
    int a=10, b=20, key=0;
    printf("a=%d,b=%d",a,d);// 使用格式化串进行输出
    return 0;
}
```

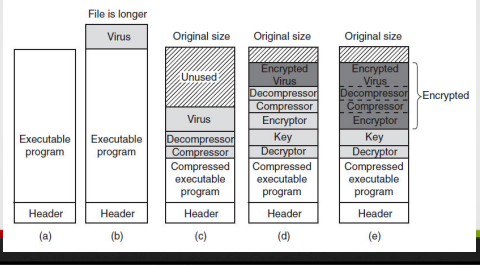


其它各种攻击
.....Too Much

感兴趣同学进一步自学

恶意软件的防御

- 病毒的攻防：加壳/脱壳



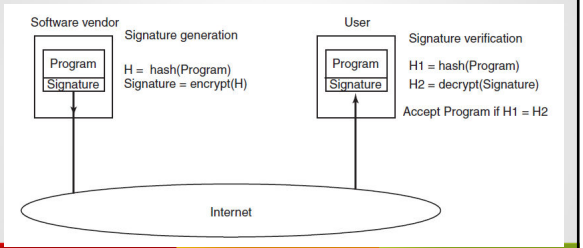
恶意软件的防御

- 病毒的攻防：多态指令/花指令

(a)	(b)	(c)	(d)	(e)
MOV A,R1	MOV A,R1	MOV A,R1	MOV A,R1	MOV A,R1
ADD B,R1	NOP	ADD #0,R1	OR R1,R1	TST R1
ADD C,R1	ADD B,R1	ADD B,R1	ADD B,R1	ADD C,R1
SUB #4,R1	NOP	OR R1,R1	MOV R1,R5	MOV R1,R5
MOV R1,X	ADD C,R1	ADD C,R1	ADD C,R1	ADD B,R1
	NOP	SHL #0,R1	SHL R1,0	CMP R2,R5
	SUB #4,R1	SUB #4,R1	SUB #4,R1	SUB #4,R1
	NOP	JMP .+1	ADD R5,R5	JMP .+1
	MOV R1,X	MOV R1,X	MOV R1,X	MOV R1,X
			MOV R5,Y	MOV R5,Y

恶意软件的防御

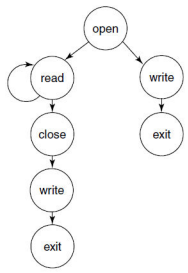
- 病毒的防御：代码签名



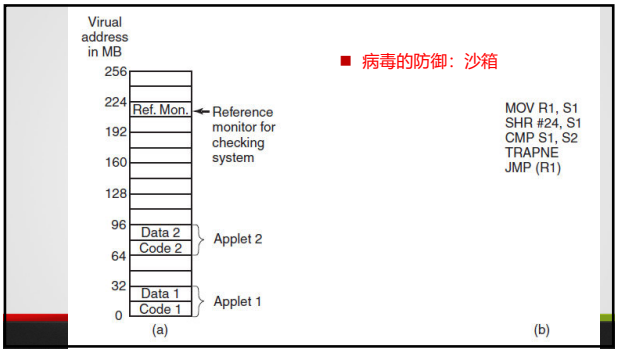
- 病毒的防御：基于模型的入侵检测(代码语义理解)

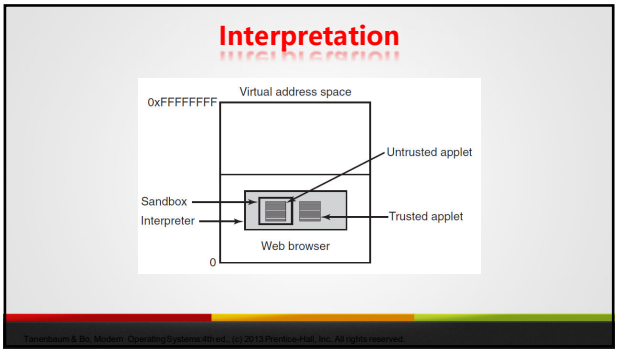
```
int main(int argc *char argv[])
{
 int fd, n = 0;
 char buf[1];

 fd = open("data", 0);
 if (fd < 0) {
    printf("Bad data file\n");
    exit(1);
 } else {
    while (1) {
       read(fd, buf, 1);
       if (buf[0] == 0) {
         close(fd);
         printf("n = %d\n", n);
         exit(0);
       }
       n = n + 1;
    }
 }
}
```

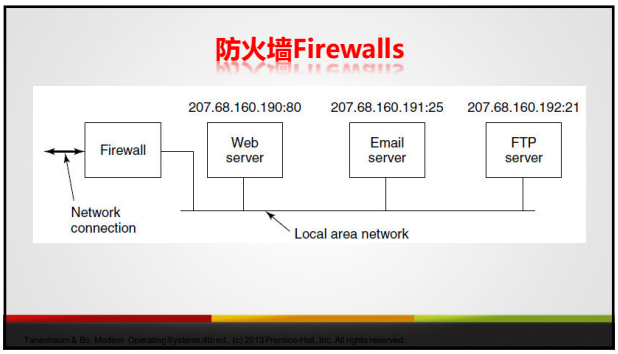
(a)



(b)


Virual address in MB
256
224
192
160
128
96
64
32
0
Ref. Mon.
Reference monitor for checking system
Data 2
Code 2
Applet 2
Data 1
Code 1
Applet 1
(a)
■ 病毒的防御：沙箱
MOV R1, S1
SHR #24, S1
CMP S1, S2
TRAPNE
JMP (R1)
(b)


Interpretation
Virtual address space
0xFFFFFFFF
Untrusted applet
Sandbox
Interpreter
Trusted applet
Web browser
0
Tanenbaum & Bo, Modern Operating Systems 4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.


防火墙Firewalls
207.68.160.190:80
207.68.160.191:25
207.68.160.192:21
Firewall
Web server
Email server
FTP server
Network connection
Local area network
Tanenbaum & Bo, Modern Operating Systems 4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.